

LECTURE 12

Inventory, Items & Player Systems

Darkwood Descent — First-Person Survival Horror

Student Guide

LECTURE INFORMATION

Course: Unreal Engine 5 Mastery — From Beginner to Game Developer

Duration: 3 hours

Project: Darkwood Descent (First-Person Survival Horror)

Part 2 of 2 — Lecture 12 of 15

Prerequisites: Lecture 11 completed, FPS character with interaction system and flashlight

Difficulty: Intermediate

Learning Objectives

By the end of this lecture, you will be able to:

1. Design data structures for items using Structs, Enums, and Data Tables
2. Build a grid-based inventory UI with drag-and-drop and item tooltips
3. Implement multiple item types: consumables, key items, documents, and weapons
4. Create Health, Stamina, and Sanity player stat systems with UI display
5. Build a sanity mechanic that distorts visuals and audio when the player is in darkness
6. Implement a basic crafting system for combining items

RECOMMENDED FREE ASSETS FOR THIS LECTURE

FAB (fab.com):

- Horror UI Kit — Dark-themed inventory slot textures and icons
- RPG Inventory Icons — Potion, herb, key, and document icons (retexture to horror palette)
- Ambient Sound Pack — Heartbeat, breathing, whisper loops for sanity effects

FREESOUND.ORG:

- Search “heartbeat loop” — For low-health audio feedback
- Search “whisper horror” — For low-sanity hallucination audio
- Search “paper rustling” — For document reading sound effects

CUSTOM (Create Yourself):

- 5–10 item icons as 128×128 PNG files (herb, battery, key, note, bandage, lockpick)

Part 1: Item Data Architecture

1.1 Designing the Item System

Before writing any Blueprint logic, we need to design the data structures that describe every item in the game. A well-designed data architecture makes it easy to add new items, balance existing ones, and keep the inventory system clean. We will use three core UE5 features: Enumerations for categories, Structures for item data, and Data Tables for storing all item definitions.

REAL GAME EXAMPLE: Resident Evil Village

Resident Evil Village categorizes every item by type: consumables (herbs, medicine), key items (crests, keys), treasures (collectibles for selling), weapons, and ammunition. Each item has a name, icon, description, stack limit, and grid size. Our system mirrors this design using UE5 Structs and Enums, giving us the same flexibility to create dozens of unique items from a single data definition.

1.2 The Item Type Enumeration

Create a new Enumeration asset in Blueprints/Data/ called E_ItemType:

Enum Value	Display Name	Behavior	Example Items
Consumable	Consumable	Used once, then removed from inventory	Herb, Bandage, Battery, Antidote
KeyItem	Key Item	Cannot be dropped or consumed; used on specific locks/puzzles	Mansion Key, Crest Piece, Fuse
Document	Document	Opens a readable text UI when used; stays in inventory	Patient Note, Diary Page, Map Fragment
Weapon	Weapon	Equippable; has durability or ammo; occupies special slot	Crowbar, Pistol, Pipe Wrench
Craftable	Crafting Material	Used as ingredient in crafting recipes	Gunpowder, Chemical Fluid, Scrap Metal

1.3 The Item Data Structure

Create a Structure asset called S_ItemData. This struct defines every property an item can have:

Variable	Type	Default	Purpose
ItemID	Name	None	Unique identifier for data table lookup

Variable	Type	Default	Purpose
DisplayName	Text	""	Shown in UI tooltip and inventory
Description	Text	""	Flavor text / gameplay hint
ItemType	E_ItemType	Consumable	Determines behavior when used
Icon	Texture2D	null	128×128 inventory slot image
bIsStackable	Boolean	true	Whether multiple copies stack in one slot
MaxStackSize	Integer	5	Maximum units per inventory slot
UseSound	Sound Cue	null	Audio played when item is used
HealthRestore	Float	0	HP restored when consumed (consumables only)
SanityRestore	Float	0	Sanity restored when consumed
BatteryRestore	Float	0	Flashlight charge restored
DocumentText	Text	""	Readable content (documents only)
RequiredKeyName	Name	None	Which lock this key opens (key items only)

KEY CONCEPT: DATA TABLES

A Data Table is a spreadsheet-like asset that stores rows of struct data. Create a Data Table with row type `S_ItemData`, then add one row per item in your game. This means designers can add, remove, and balance items without opening a single Blueprint. When the inventory needs item data, it looks up the `ItemID` in the Data Table and gets all properties instantly. This is the same approach used in production by studios like Capcom and Naughty Dog.

Example Data Table Entries

ItemID	DisplayName	Type	Stackable	MaxStack	HealthRestore	SanityRestore
Herb_Green	Green Herb	Consumable	Yes	3	25.0	0
Herb_Blue	Blue Herb	Consumable	Yes	3	0	30.0
Battery_AA	AA Battery	Consumable	Yes	5	0	0
Key_Basement	Basement Key	KeyItem	No	1	0	0
Note_Patient01	Patient Note #1	Document	No	1	0	0
Bandage	Bandage	Consumable	Yes	5	40.0	0

ItemID	DisplayName	Type	Stackable	MaxStack	HealthRestore	SanityRestore
Gunpowder	Gunpowder	Craftable	Yes	3	0	0
ChemFluid	Chemical Fluid	Craftable	Yes	3	0	0

 PRO TIP

Use a naming convention for ItemIDs: Category_Name. This makes Data Table sorting and Blueprint lookups much cleaner. Key_Basement, Note_Patient01, Herb_Green are all instantly recognizable. Avoid spaces or special characters in ItemIDs—they cause issues with Blueprint string comparisons.

Part 2: Grid-Based Inventory System

2.1 Inventory Architecture

The inventory is an array of `S_InventorySlot` structs stored on the player character. Each slot holds an `ItemID` (Name), a `Quantity` (Integer), and a reference to the full `S_ItemData` from the Data Table. The inventory has a fixed number of slots (24 for a 4×6 grid), and empty slots have an `ItemID` of "None."

KEY CONCEPT: SLOT-BASED vs GRID-SIZE INVENTORY

Resident Evil 4's attache case uses variable-size items (a pistol takes 3×2 slots, an herb takes 1×1). This is complex to implement. Resident Evil 7 and Village simplified to uniform-size slots where each item takes exactly one slot, but items can stack. We will use the simpler uniform-slot approach—it is easier to build, easier to use, and equally effective for horror resource management.

2.2 Inventory Slot Structure

Create a second Structure called `S_InventorySlot`:

Variable	Type	Default	Purpose
ItemID	Name	None	References a row in the Item Data Table
Quantity	Integer	0	How many of this item are in this slot
bIsEmpty	Boolean	true	Quick check for empty slots

2.3 Core Inventory Functions

Add these functions to `BP_HorrorCharacter` (or a dedicated `AC_Inventory` ActorComponent for cleaner architecture):

Function	Inputs	Returns	Logic
AddItem	ItemID (Name), Amount (Int)	Bool (success)	Find existing stack or first empty slot; add quantity; respect <code>MaxStackSize</code> ; return false if inventory full
RemoveItem	ItemID (Name), Amount (Int)	Bool (success)	Find slot with matching ID; subtract quantity; clear slot if quantity reaches 0
UseItem	SlotIndex (Int)	None	Look up item type; call appropriate use function (heal, read, equip); remove if consumable

Function	Inputs	Returns	Logic
DropItem	SlotIndex (Int)	None	Spawn BP_WorldItem at player feet; transfer item data; clear inventory slot
HasItem	ItemID (Name)	Bool	Search all slots for matching ID with quantity > 0
GetItemCount	ItemID (Name)	Integer	Sum quantities across all slots with matching ID
FindEmptySlot	None	Integer (-1 if full)	Loop through slots; return first index where bIsEmpty is true
SwapSlots	IndexA (Int), IndexB (Int)	None	Exchange contents of two slots (for drag-and-drop reorganization)

COMMON MISTAKE

When adding stackable items, always check MaxStackSize before increasing quantity. If the player has 3 Green Herbs in a slot (max 3) and picks up 2 more, you need to fill the existing slot to max and overflow the remaining 2 into a new slot. Many beginner implementations lose items by ignoring overflow. Write the AddItem function to handle partial stacking with a while loop that fills slots until all items are placed or inventory is full.

Part 3: Inventory UI with Widget Blueprints

3.1 Widget Hierarchy

The inventory UI consists of three Widget Blueprints that work together:

Widget	Purpose	Key Components
WBP_InventoryScreen	Full-screen overlay when inventory is open	4×6 UniformGridPanel, item detail panel, use/drop buttons, crafting button
WBP_InventorySlot	Individual slot in the grid (created 24 times)	Border (highlight on hover), Image (item icon), Text (stack count), drag-drop target
WBP_ItemTooltip	Popup showing item details on hover	Item name, type badge, description text, stats (heal amount, etc.)

3.2 Building WBP_InventorySlot

- Step 1: Create a new Widget Blueprint called WBP_InventorySlot. Set it as a User Widget.
- Step 2: Root: a Size Box set to 80×80 pixels. This ensures uniform grid cells.
- Step 3: Child 1: a Border widget with dark gray background (#1A1A2E) and a 1px border (#444444). This is the slot frame.
- Step 4: Child 2: an Image widget filling the border—this displays the item icon. When the slot is empty, set it to a transparent placeholder or dim grid texture.
- Step 5: Child 3: a Text Block anchored bottom-right showing the stack count (e.g., “x3”). Hide when quantity is 1 or slot is empty.
- Step 6: Add a variable SlotIndex (Integer, Instance Editable) so the parent grid knows which slot this widget represents.
- Step 7: Implement On Mouse Enter: show highlight border (amber #F9A825) and trigger WBP_ItemTooltip display. On Mouse Leave: revert border color and hide tooltip.
- Step 8: Implement On Mouse Button Down (left click): call UseItem on the inventory. Right click: open context menu (Use / Drop / Examine).

3.3 Drag and Drop

UE5 provides built-in drag-and-drop support for UMG widgets. This lets players reorganize their inventory by dragging items between slots.

- Step 9: On WBP_InventorySlot, override On Mouse Button Down to call Detect Drag If Pressed (Left Mouse Button).
- Step 10: Override On Drag Detected: create a Drag Drop Operation. Set the Payload to the SlotIndex. Create a visual: a semi-transparent copy of the item icon that follows the cursor.

- Step 11: Override On Drop on WBP_InventorySlot: read the Payload (source SlotIndex), call SwapSlots(SourceIndex, ThisSlotIndex) on the inventory, and refresh both slot widgets.
- Step 12: Override On Drag Cancelled: return the visual to the original slot with no changes.

PRO TIP

For horror atmosphere, use a dark, muted color palette for the inventory UI. Resident Evil Village uses black backgrounds with off-white text and minimal color. Avoid bright, saturated UI elements—they break immersion. Add a subtle background blur or darken effect on the game world when the inventory is open, but keep the game running (not paused) so the player feels vulnerable.

REAL GAME EXAMPLE: Dead Space

Dead Space projects the inventory as a holographic display in front of the character. The game does not pause—enemies can still attack while you manage items. This creates incredible tension. In Darkwood Descent, we will similarly keep the game unpaused when the inventory is open. The player must find a safe moment to manage their items, adding another layer of survival pressure.

Part 4: Player Stat Systems

4.1 Health System

The health system tracks damage from environmental hazards, enemy attacks, and traps. Unlike Shadow Ops (which used a regenerating armor system), Darkwood Descent uses classic survival horror health—damage is permanent until healed with items.

Property	Type	Value	Notes
MaxHealth	Float	100.0	Does not increase during gameplay
CurrentHealth	Float	100.0	Decreased by Apply Damage, increased by consumables
bIsDead	Boolean	false	Set true when CurrentHealth reaches 0
LowHealthThreshold	Float	25.0	Below this: red vignette, heartbeat audio, screen pulse
CriticalHealthThreshold	Float	10.0	Below this: screen desaturates, movement slowed 20%

Health UI Feedback

Rather than a visible health bar (which breaks immersion in first-person horror), we communicate health through environmental feedback:

Health Range	Visual Effect	Audio Effect
100–75%	No effect — player feels safe	Normal ambient sounds
75–50%	Subtle blood spatters on screen edges	Slightly heavier breathing
50–25%	Red vignette pulses gently every 3 seconds	Audible heartbeat (slow), pained grunts on movement
25–10%	Strong red vignette, screen desaturation begins	Fast heartbeat, heavy breathing, movement slowed
Below 10%	Screen nearly monochrome, heavy blood overlay	Pounding heartbeat, distorted audio, limping footsteps

KEY CONCEPT: DIEGETIC UI

Diegetic UI means the interface exists within the game world rather than as an overlay. Instead of a health bar, the player sees blood on their hands, hears their character's labored breathing, and notices the world losing color. This approach, used by Dead Space, Alien: Isolation, and Metro Exodus, keeps the player immersed in the horror. We will use post-process material parameters driven by the CurrentHealth variable to create these effects.

4.2 Stamina System

Stamina governs the player's ability to sprint and perform strenuous actions. It regenerates automatically but slowly, encouraging careful resource management:

Property	Type	Value	Notes
MaxStamina	Float	100.0	Full stamina bar
CurrentStamina	Float	100.0	Drained by sprinting and heavy actions
SprintDrainRate	Float	15.0/sec	Depletes in ~6.5 seconds of continuous sprint
RegenRate	Float	8.0/sec	Only regenerates when not sprinting (1.5s delay after stopping)
RegenDelay	Float	1.5 sec	Pause before regeneration begins after stamina use
ExhaustedThreshold	Float	0.0	When stamina hits 0, player cannot sprint for 3 seconds

DESIGN WARNING

Do not make stamina regeneration instant or unlimited sprint possible. The entire tension of a chase sequence (Lecture 14) relies on the player running out of stamina and needing to hide. If the player can sprint indefinitely, the monster AI becomes a minor inconvenience rather than a terrifying threat. Test your drain and regen rates by sprinting through your test level—the player should run out of stamina before reaching the far end.

4.3 Sanity System

The Sanity system is the signature mechanic of Darkwood Descent, directly inspired by Amnesia: The Dark Descent. The player's mental state deteriorates when they spend time in darkness, witness disturbing events, or remain near the monster. Low sanity triggers increasingly severe visual and audio hallucinations.

REAL GAME EXAMPLE: Amnesia: The Dark Descent

Amnesia's sanity mechanic made darkness itself the enemy. Standing in the dark drained sanity, causing the screen to warp, insects to crawl across the view, and false sounds to play. Players had to balance staying in light (which attracted the monster) against staying in darkness (which destroyed their sanity). Our system recreates this exact tension.

Sanity Properties

Property	Type	Value	Notes
MaxSanity	Float	100.0	Full mental health
CurrentSanity	Float	100.0	Core sanity value
DarknessDrainRate	Float	3.0/sec	Drain when in darkness (no nearby light source)
MonsterDrainRate	Float	8.0/sec	Drain when monster is within 15 meters
LightRegenRate	Float	2.0/sec	Regenerates slowly when standing in lit area
ItemRestoreAmount	Float	30.0	Blue Herb or Sanity Pills restore this amount

Detecting Darkness

To determine whether the player is in darkness, we use a clever technique:

- Step 13: Attach a Scene Capture Component 2D to the player at head level, rendering a tiny 1×1 pixel texture.
- Step 14: On Tick, read the single pixel's luminance value from the render target.
- Step 15: If the luminance is below a threshold (e.g., 0.05), the player is considered to be in darkness and sanity begins draining.
- Step 16: Alternatively, use a simpler approach: perform multiple line traces outward from the player to detect nearby light actors within radius. If no light sources are within 500 units, the player is in darkness.

PRO TIP

The line-trace-to-lights approach is much cheaper on performance than scene capture. For most horror games, checking distance to the nearest Point Light or Spot Light actor every 0.5 seconds (not every frame) is sufficient. The player's own flashlight counts as a light source—this creates the resource tension between battery conservation and sanity preservation.

Sanity Effect Levels

Sanity Range	Visual Effects	Audio Effects	Gameplay Effects
100–75%	None	None	None
75–50%	Subtle screen wobble (sine wave, 0.5° amplitude)	Distant whispers at low volume	None
50–25%	Chromatic aberration increases, periodic screen distortion	Whispers louder, occasional phantom footsteps	Mouse sensitivity fluctuates $\pm 5\%$
25–10%	Heavy visual noise, color shifts, shadow figures flash briefly	Constant whispers, false door sounds, breathing audio	Interaction prompts flicker unreliably
Below 10%	Screen pulses, extreme distortion, hallucination flashes	Screaming, heartbeat, audio cuts in/out	Random stumble (brief movement freeze)

COMMON MISTAKE

Do not make sanity effects so extreme that the game becomes unplayable. The distortions should be unsettling, not nauseating. Test each tier individually and get feedback from playtesters. Chromatic aberration above 2.0 and heavy screen shake can cause motion sickness. Subtle, psychological effects (whispers, shadow flickers, unreliable UI) are more effective than aggressive screen distortion.

Part 5: Document & Collectible System

5.1 Reading Documents

Documents are the primary storytelling tool in survival horror. Patient notes, diary entries, research logs, and warning messages deliver lore, hints, and atmosphere. When the player uses a document from their inventory, a full-screen readable UI appears.

- Step 17: Create WBP_DocumentReader: a full-screen widget with a dark, parchment-like background texture.
- Step 18: Add a Scroll Box containing a Rich Text Block. The Rich Text Block supports bold, italic, and color markup for emphasis in document text.
- Step 19: Add a title at the top (document's DisplayName), a close button (X or press E), and a paper rustling sound that plays when the widget opens.
- Step 20: When UseItem is called on a Document type, create the WBP_DocumentReader widget, populate it with the DocumentText field from S_ItemData, and set Input Mode to UI Only.
- Step 21: On close, return to Game Input Mode. Mark the document as "read" in a separate array so the player can track which notes they have found.

5.2 Collection Tracking

Add an array of Names called CollectedDocuments to the player character. When a document is first read, add its ItemID to this array. Create a WBP_JournalScreen that displays all collected documents in a list, letting the player re-read any note at any time. Show collected count (e.g., "7 / 15 Documents Found") to encourage exploration.

REAL GAME EXAMPLE: Outlast / The Forest

Outlast tracks documents in a journal that the player can review at any time. Each document has a thumbnail and title. The Forest displays notes pinned to a physical journal that the character holds. Both approaches encourage thorough exploration. In Darkwood Descent, documents also serve as puzzle hints—a note might mention "the key is behind the painting in the study," guiding observant players.

Part 6: Crafting System

6.1 Crafting Design

The crafting system lets players combine two items to create a more useful result. This is inspired directly by Resident Evil 7's combine mechanic, where mixing a Chemical Fluid with Gunpowder creates handgun ammo, or combining two herbs creates a stronger healing item.

6.2 Recipe Data Structure

Create a Structure called `S_CraftingRecipe` and a Data Table `DT_CraftingRecipes`:

Variable	Type	Purpose
InputA	Name (ItemID)	First ingredient
InputB	Name (ItemID)	Second ingredient
OutputItem	Name (ItemID)	Resulting item
OutputQuantity	Integer	How many of the result to produce

Example Recipes

Input A	Input B	Output	Qty	Gameplay Value
Herb_Green	Herb_Green	StrongMedicine	1	Heals 60 HP instead of 25
Herb_Green	Herb_Blue	MixedHerb	1	Heals 25 HP + restores 30 Sanity
Gunpowder	ChemFluid	PistolAmmo	5	Creates ammunition for pistol
ScrapMetal	ScrapMetal	Lockpick	1	Opens locked drawers and cabinets
Bandage	Herb_Green	MedicKit	1	Heals 75 HP (best healing item)

6.3 Crafting Implementation

- Step 22: Create a `CraftItems` function that takes two `SlotIndex` values as input.
- Step 23: Look up both slots' `ItemIDs`. Search `DT_CraftingRecipes` for a row where `InputA` and `InputB` match (check both orders—`A+B` and `B+A` should both work).
- Step 24: If a matching recipe is found: remove 1 quantity of each input item, then call `AddItem` with the `OutputItem` and `OutputQuantity`.
- Step 25: If no recipe matches: display a brief "Cannot combine" message on screen and play a failure sound.
- Step 26: In the inventory UI, add a "Combine" mode: the player clicks one item, then clicks another, and the `CraftItems` function is called with both slot indices.

 PRO TIP

Add visual feedback for valid combinations. When the player selects the first item in Combine mode, highlight any other items that can combine with it by adding a subtle glow border on valid partner slots. This prevents frustration from trial-and-error and is how Resident Evil Village handles it.

Hands-On Exercises

EXERCISE 1: Item Data Architecture (20 minutes)

Create `E_ItemType` enumeration with all five values (Consumable, KeyItem, Document, Weapon, Craftable).

Create `S_ItemData` structure with all 13 variables listed in Part 1.

Create a Data Table `DT_Items` using `S_ItemData` as the row type.

Add at least 8 items: 2 consumables (Herb, Bandage), 2 key items (Basement Key, Study Key), 2 documents (Patient Note, Diary Page), 1 crafting material (Gunpowder), and 1 battery.

Verify: open the Data Table and confirm all rows display correct data with icons assigned.

EXERCISE 2: Inventory System and UI (30 minutes)

Create `S_InventorySlot` structure with `ItemID`, `Quantity`, and `IsEmpty`.

Add an `InventorySlots` array (24 elements) to `BP_HorrorCharacter`. Initialize all as empty.

Implement `AddItem`, `RemoveItem`, `HasItem`, and `FindEmptySlot` functions with stack overflow handling.

Build `WBP_InventorySlot` (80×80, dark background, icon image, stack count text).

Build `WBP_InventoryScreen` with a 4×6 `UniformGridPanel` populated with 24 `WBP_InventorySlot` widgets.

Bind the Tab key to toggle inventory visibility. Test: pick up items from the world and verify they appear in the grid.

EXERCISE 3: Health, Stamina & Sanity (25 minutes)

Add Health, Stamina, and Sanity variables with all properties from the tables in Part 4.

Implement stamina drain during sprint (15/sec) with 1.5s regen delay and 8/sec regeneration.

Implement sanity drain: check distance to nearest light actor every 0.5 seconds. If no light within 500 units, drain at 3.0/sec.

Add a Post Process Dynamic Material Instance. Drive Vignette intensity, desaturation, and chromatic aberration from `CurrentHealth` and `CurrentSanity`.

Test: stand in darkness and watch sanity drain. Use a Blue Herb to restore sanity. Sprint until exhausted and verify the 3-second lockout.

EXERCISE 4: Documents and Crafting (15 minutes)

Create `WBP_DocumentReader` with dark background, Rich Text Block, and close button.

Wire the UseItem function to display WBP_DocumentReader when a Document type item is used.

Create S_CraftingRecipe and DT_CraftingRecipes with at least 3 recipes from the table in Part 6.

Implement CraftItems function with bidirectional recipe lookup (A+B and B+A).

Test: combine two Green Herbs to create Strong Medicine. Verify inputs are consumed and output appears in inventory.

Self-Assessment Checklist

Check off each item confidently before moving to Lecture 13.

Topic	Skill	☐
Data	I created E_ItemType, S_ItemData, and a Data Table with 8+ items	
Data	I understand how Data Table row lookups work with ItemID as key	
Inventory	I built AddItem with proper stack overflow handling	
Inventory	RemoveItem, HasItem, DropItem, and SwapSlots all function correctly	
UI	WBP_InventorySlot displays icon, stack count, and hover highlight	
UI	WBP_InventoryScreen shows a 4×6 grid that refreshes when inventory changes	
UI	Drag-and-drop item reorganization works between slots	
Health	Damage reduces health; consumables restore it; diegetic feedback works	
Stamina	Sprint drains stamina; regeneration has a 1.5s delay; exhaustion locks sprint	
Sanity	Sanity drains in darkness and near monsters; visual/audio effects scale by tier	
Sanity	The player's flashlight counts as a light source for sanity calculations	
Documents	Using a document opens WBP_DocumentReader with correct text	
Crafting	Combining two valid items consumes inputs and produces output	
Crafting	Invalid combinations show a failure message without consuming items	

Lecture Deliverables

DELIVERABLES BY END OF LECTURE

-  E_ItemType enumeration and S_ItemData structure with 13 properties
-  DT_Items Data Table populated with 8+ unique items
-  24-slot grid inventory with AddItem, RemoveItem, UseItem, DropItem, SwapSlots
-  WBP_InventoryScreen with hover tooltips and drag-and-drop
-  Health system with diegetic feedback (vignette, desaturation, heartbeat)
-  Stamina system with sprint drain, regen delay, and exhaustion lockout
-  Sanity system with darkness detection and tiered visual/audio effects
-  WBP_DocumentReader for readable notes with collection tracking
-  Crafting system with recipe Data Table and bidirectional lookup

Next Lecture Preview

Lecture 13: Horror Atmosphere — Lighting, Audio & Effects

In the next lecture, we will transform the Darkwood Descent mansion into a genuinely terrifying environment. We will master horror lighting with practical light sources and dynamic shadows, build a layered ambient audio system with 3D positional sound, create an adaptive music manager that responds to gameplay tension, and add post-processing effects that make every room feel oppressive and dread-inducing.

PREPARATION TASKS FOR LECTURE 13

1. Download ambient audio packs: wind, rain, wood creaking, distant thunder (Freesound.org or Fab).
2. Download horror music assets: tension drones, stinger sounds, exploration loops (Fab: “Horror Music Pack”).
3. Collect reference images from Amnesia, Resident Evil 7, and Outlast showing lighting setups.
4. Build 3–4 additional rooms connected by hallways to your test room from Lecture 11. Use BSP geometry—we will dress them with proper lighting in Lecture 13.
5. Review Lumen Global Illumination settings from Lecture 1—we will fine-tune them for dark interiors.

WHAT WE'RE BUILDING TOWARD

By the end of Lecture 13, your mansion will have oppressive horror lighting with only practical light sources, volumetric fog that reacts to your flashlight, 3D audio where you hear creaking floorboards from specific directions, and adaptive music that shifts from eerie exploration drones to heart-pounding tension when danger is near—just like *Amnesia: The Dark Descent* and *Resident Evil 7*.